Ejercicio (OrganziadorAula.java)

Cada alumno entrega sus tareas como archivos sueltos en una carpeta "Descargas". Quieres **organizar automáticamente** esos archivos en una carpeta del curso, creando la estructura si no existe.

Programa en Java que:

1. **Cree** (si no existen) las carpetas:
    - CursoDAM/Entregas
2. Dentro de Entregas, **cree un fichero** README.txt con el texto:
    - Coloca aquí las entregas de los alumnos.
3. **Muestre por consola**:
    - Ruta absoluta de CursoDAM
    - Si existen CursoDAM, Entregas y README.txt
4. (Opcional) Si ya existen, **no borres nada**: solo informa de su existencia.

Pista: usa java.io.File para crear carpetas y ficheros y para comprobar su existencia.

Ejercicio (GestorEntregasMedio.java)

Se desea estandarizar cómo se organizan las entregas del alumnado por asignaturas, dejando constancia de la **fecha de inicialización**, y obtener un **inventario** de lo creado para revisar que todo está en su sitio.

Programa en Java que:

1. **Crea la estructura base**, si no existe:

```
CursoDAM/
  Entregas/
    AccesoDatos/
    Programacion/
    Interfaces/
```

2. En cada carpeta de asignatura crea un **README.txt** con este contenido:

```
Carpeta de entregas para <ASIGNATURA>.
Inicializada el <FECHA-YYYY-MM-DD> a las <HORA-HH:MM>.
```

3. Genera en CursoDAM/Entregas/ un fichero **inventario.txt** que liste:
    - Cada asignatura, su ruta absoluta.
    - Si README.txt existe y su **tamaño en bytes**.

    - Número de ficheros actualmente en cada carpeta de asignatura (sin contar el README).
4. **No sobrescribas** lo ya existente (idempotente): si una carpeta/README ya existe, se mantiene y solo se informa.
5. Valida:
    - Si alguna asignatura **no se pudo crear**, informar al final con un resumen de errores.
    - Si no hay permisos de escritura, mostrar un mensaje claro y **no interrumpir** el resto de tareas.

Pistas:

- File.mkdir() / mkdirs(), listFiles(), length(), getAbsolutePath().
- LocalDate, LocalTime (o LocalDateTime) para fecha/hora.
- Para idempotencia: comprobar exists() antes de crear/escribir.

Ejercicio (GestorEntregasAlto)

Desarrolla una utilidad en Java que:

1. Reciba la **lista de asignaturas** por **argumentos** (args) o, si no hay argumentos, las **lea de un fichero** asignaturas.txt (una por línea).
2. Cree (si no existiera) la estructura:

```
CursoDAM/
  Entregas/
    <ASIGNATURA>/
      README.txt
```

   El README.txt incluirá:

```
Carpeta de entregas para <ASIGNATURA>.
Inicializada el <YYYY-MM-DD> a las <HH:MM>.
```

3. Genere en CursoDAM/Entregas/:
    - inventario.txt con:
        - Ruta absoluta de cada carpeta de asignatura.
        - Existencia de README y **tamaño** de ese README.
        - **Número de ficheros** (excluyendo README).
        - **Tamaño total** (suma de ficheros de la carpeta, sin README).

- inventario.csv con columnas: asignatura,ruta_absoluta,readme_existe,tam_readme_bytes,num_ficheros,tam_total_bytes

2. **No sobrescriba** recursos si ya existen (idempotente). Recalcula siempre el inventario.
3. Maneje errores sin detener la ejecución completa y muestre un **resumen final** de incidencias.

Requisitos técnicos: usar NIO.2 (Path, Files, StandardOpenOption, DirectoryStream, etc.)

Ejemplos simples con NIO.2

Crear un directorio con `java.io.File` (antiguo):

```
File carpeta = new File("datos");
if (!carpeta.exists()) {
    carpeta.mkdir();
}
```

Crear un directorio con NIO.2 (`Path` + `Files`):

```
import java.nio.file.*;

Path carpeta = Paths.get("datos");
if (Files.notExists(carpeta)) {
    Files.createDirectory(carpeta);
}
```

Leer todas las líneas de un fichero (NIO.2)

```
import java.nio.file.*;
import java.io.IOException;
import java.util.List;

public class LeerLineasNIO2 {
    public static void main(String[] args) throws IOException {
        Path ruta = Paths.get("alumnos.txt");
        List<String> lineas = Files.readAllLines(ruta);
        for (String l : lineas) {
            System.out.println(l);
        }
    }
}
```

Copiar un fichero (NIO.2)

```
Path origen = Paths.get("imagen.jpg");
Path destino = Paths.get("copia.jpg");
Files.copy(origen, destino,
StandardCopyOption.REPLACE_EXISTING);
```